

TechMentor-LLM

Complete Project Guide

Dataset Engineering, QLoRA Fine-Tuning,
and Interview Preparation

Generated: 2026-06-25

Purpose: Understand every part of the codebase and explain it confidently in technical interviews.

1. Executive Summary (30-Second Interview Pitch)

TechMentor-LLM is an end-to-end GenAI project that builds a domain-specific AI interview mentor. It ingests three public datasets (interview conversations, technical Q&A, and code reasoning), cleans and unifies them, converts them into instruction-tuning examples, validates quality, and fine-tunes Llama 3.2 3B Instruct using QLoRA so the model can answer interview questions, generate follow-ups, explain code, and evaluate candidate responses.

The project demonstrates practical MLOps and LLM engineering: data pipelines, dataset versioning with Weights & Biases, parameter-efficient fine-tuning (PEFT/LoRA), 4-bit quantization, experiment tracking, and structured evaluation.

Key numbers to remember

Metric	Value
Raw records ingested	24,242
Clean unified records	7,928
Final instruction examples	560 (504 train / 56 eval)
Base model	Llama 3.2 3B Instruct
Fine-tuning method	QLoRA (4-bit + LoRA rank 16)
Trainable params reduction	>99% vs full fine-tune

2. Problem Statement and Solution Design

General-purpose LLMs know broad programming concepts but are not optimized for structured technical interview mentoring. TechMentor solves this with supervised fine-tuning (SFT) on a curated multi-task instruction dataset.

What the model should do

- Answer technical interview questions clearly.
- Generate follow-up interview questions.
- Explain code snippets.
- Evaluate candidate answers with structured feedback.
- Support backend, SQL, ML, and system design topics.

Design decisions

- Use instruction tuning (Alpaca format) instead of raw text completion.
- Mix multiple task types (70/10/10/10) to avoid single-task overfitting.
- Use QLoRA to train on consumer GPUs (e.g., Colab T4).
- Keep LoRA adapter separate from base model for flexibility and smaller artifacts.

3. End-to-End Architecture

Pipeline flow: Download -> Clean/Unify -> Convert to Alpaca -> Validate -> Fine-tune -> Infer/Evaluate.

```
data/raw/ --> clean_data.py --> data/cleaned/  
data/cleaned/ --> convert_to_alpaca.py --> data/processed/train.jsonl + eval.jsonl  
data/processed/ --> validate_dataset.py --> reports/ + W&B artifact  
data/processed/ --> train_unsloth.py --> training/outputs/final_adapter/  
final_adapter/ --> inference.ipynb + generate_comparison.py --> evaluation/before_after.md
```

Folder responsibilities

Path	Role
data/raw/	Original downloaded datasets
data/cleaned/	Unified schema, deduplicated records
data/processed/	Alpaca JSONL ready for SFT
reports/	Stats, charts, sample records
training/	Fine-tuning code, configs, outputs
evaluation/	Base vs fine-tuned comparisons
notebooks/	EDA and dataset reporting

4. Core ML and LLM Concepts You Must Explain

4.1 Instruction Tuning

Instruction tuning teaches a model to follow task instructions. Each training example has three fields: instruction (what to do), input (context/question), and output (desired response). This is more controllable than continuing raw text.

4.2 Alpaca Format

```
{  
  "instruction": "Answer the technical interview question.",  
  "input": "What is dependency injection?",  
  "output": "Dependency injection is a design pattern..."  
}
```

4.3 LoRA (Low-Rank Adaptation)

Instead of updating all billions of model weights, LoRA inserts small trainable matrices into attention and MLP layers. Only these adapter weights are updated. This reduces memory, speeds training, and prevents catastrophic forgetting of general knowledge.

4.4 QLoRA (Quantized LoRA)

QLoRA combines 4-bit quantized base model weights (via BitsAndBytes) with LoRA adapters trained in higher precision. You get near full fine-tune quality with much lower GPU memory.

4.5 PEFT

PEFT (Parameter-Efficient Fine-Tuning) is the Hugging Face library abstraction for methods like LoRA. It manages adapter injection, saving, and loading.

4.6 Unsloth

Unsloth is an optimized training stack on top of Transformers/TRL that speeds up LoRA training and reduces memory via kernel optimizations and efficient gradient checkpointing.

4.7 SFTTrainer (TRL)

Supervised Fine-Tuning Trainer from TRL handles tokenization, loss computation on assistant tokens, evaluation loops, checkpointing, and logging integration.

4.8 Weights & Biases (W&B)

W&B tracks experiments: hyperparameters, metrics (train/eval loss), artifacts (dataset versions), and run lineage for reproducibility.

5. Data Sources and Why Each Was Chosen

Source	File	Records	Purpose
HF Interview	hf_interview.json	~200	Conversation flow, follow-ups
Kaggle SE Q&A	kaggle_questions.csv	~200	Categorized Q&A with difficulty
CodeQA paper	codeqa.json	~76k	Code explanation and Python reasoning

Unified schema (all sources)

```
question, answer, candidate_answer, followup, code,
category, difficulty, source
```

Unused fields are null. This schema lets one cleaning pipeline handle heterogeneous sources and merge complementary fields when questions overlap.

6. Code Walkthrough: download_data.py

Location: data/scripts/download_data.py. Downloads all raw datasets into data/raw/.

- HF dataset loaded via pandas read_parquet from hf:// URI (Hugging Face + fsspec).
- Kaggle dataset downloaded with kagglehub; CSV re-encoded from Latin-1 to UTF-8.
- CodeQA loaded from HF mirror; columns renamed to question/code/answer.

Outputs

```
data/raw/hf_interview.json
data/raw/kaggle_questions.csv
data/raw/codeqa.json
```

7. Code Walkthrough: clean_data.py

This is the data engineering core. It standardizes, filters, deduplicates, and merges records.

Loading logic

- HF: extracts question from first assistant turn; follow-up from second assistant turn.
- Kaggle: maps Question/Answer/Category/Difficulty columns.
- CodeQA: rejects short answers (<5 words or <30 chars) early.

Cleaning rules

- Remove null/empty questions.
- Remove answers with fewer than 5 words (when answer exists).
- Deduplicate by question.lower().strip().
- On duplicate: merge fields (HF follow-up + Kaggle answer/category + CodeQA code).
- Normalize whitespace and map categories (e.g., machine_learning -> ml).

Important interview point

Merging on duplicate questions preserves signal from all sources instead of dropping valuable fields. Example: same question in HF and Kaggle keeps follow-up from HF and reference answer from Kaggle.

8. Code Walkthrough: `convert_to_alpaca.py`

Transforms cleaned records into four instruction task types and splits train/eval.

Task	Share	Instruction
Interview QA	70%	Answer the technical interview question.
Follow-up	10%	Generate a follow-up interview question.
Code explanation	10%	Explain the following code.
Interview evaluation	10%	Evaluate the candidate answer.

- Dataset size is anchored by follow-up pool size (limited HF conversations).
- `train_test_split(test_size=0.1, random_state=42)` for reproducibility.
- Outputs JSONL lines for efficient streaming during training.

9. Code Walkthrough: `validate_dataset.py` (Day 3)

Pre-training quality gate. Ensures Alpaca rows are valid and generates EDA artifacts.

- Step 1: Load `train.jsonl` and `eval.jsonl`.
- Step 2: Assert each row has instruction, input, output; output length > 10 chars.
- Step 3: Update `dataset_stats.json` with train/eval/total sample counts.
- Step 4: Plot category distribution from `cleaned_dataset.json`.
- Step 5: Plot answer length histogram from output token/word counts.
- Step 6: Detect duplicate inputs, instructions, and outputs.
- Step 7: Write 20 random samples to `sample_records.md` for manual QA.
- Step 8-9: Optional W&B init + dataset artifact (train, eval, stats, charts).

Why validation before training?

Training on malformed data wastes GPU hours and produces unstable loss curves. Validation catches empty outputs, schema drift, and duplicate leakage between train and eval early.

10. Fine-Tuning Stack (Day 4)

`config.py`

Central hyperparameters and paths. Defines `LoRAConfig` (`r=16`, `alpha=32`, `dropout=0.05`) and `TrainingConfig` (`epochs=2`, `lr=2e-4`, `batch=2`, `grad_accum=4`, `max_seq_len=2048`).

prompts.py

Converts Alpaca triplets into Llama 3.2 chat template using `tokenizer.apply_chat_template`. System role carries instruction; user role carries input; assistant role carries target output.

utils.py

- Loads JSONL and builds Hugging Face Dataset with a text field.
- Saves `training_config.json`, `lora_config.json`, `wandb_run_url.txt`.
- Saves adapter + tokenizer to `final_adapter/`.

train_unsloth.py (primary)

- Loads 4-bit quantized Llama 3.2 via `FastLanguageModel`.
- Injects LoRA adapters into attention/MLP projections.
- Trains with TRL SFTTrainer and logs to W&B.
- Supports `--dry-run` and `--no-wandb` flags.

train_hf.py (fallback)

Same training flow using native Transformers + PEFT + BitsAndBytesConfig for environments without Unsloth.

Training hyperparameter rationale

Hyperparameter	Value	Why
LoRA rank (r)	16	Balance capacity vs overfitting on small dataset
LoRA alpha	32	Scaling factor (typically 2x rank)
Learning rate	2e-4	Common effective LR range for LoRA SFT
Epochs	2	Enough adaptation without memorization
Batch size	2	Fits T4 memory with 4-bit model
Grad accumulation	4	Effective batch size = 8
Max seq length	2048	Covers long code explanation samples
Warmup ratio	0.03	Stabilizes early training steps

Llama 3.2 chat formatting

Before tokenization, each Alpaca example becomes a chat transcript. During training, the model learns to produce assistant messages given system+user context.

System: Answer the technical interview question.

User: What is JWT?

Assistant: JWT stands for JSON Web Token...

generate_comparison.py

Loads base 4-bit model and fine-tuned PeftModel adapter separately (fair comparison). Runs six fixed test prompts and writes evaluation/before_after.md with side-by-side outputs.

Why not merge adapter into base model yet?

Keeping adapter separate enables smaller artifacts, easier A/B testing, swapping adapters per task, and safer deployment experimentation before full merge/export.

11. Inference and Evaluation

- inference.ipynb: loads base model + LoRA adapter, runs test prompts.
- generate_comparison.py: compares base vs fine-tuned outputs.
- evaluation/before_after.md: side-by-side results for README and demos.

Six evaluation prompt categories

- QA: Explain dependency injection.
- Backend: JWT vs OAuth.
- SQL: Database indexing.
- ML: Bagging vs boosting.
- Code explanation: Python merge function.
- Interview evaluation: Score a weak JWT answer.

11b. Day-by-Day Project Timeline

Day	Focus	Key Outputs
Day 1-2	Data engineering	raw/, cleaned/, processed/
Day 3	Validation + W&B	reports/, dataset artifact
Day 4	QLoRA fine-tuning	final_adapter/, W&B run

11c. Technologies Used

Layer	Tools
Language	Python 3.12, uv
Data	pandas, scikit-learn, kagglehub
ML/LLM	Transformers, TRL, PEFT, BitsAndBytes, Unsloth
Model	meta-llama/Llama-3.2-3B-Instruct
Tracking	Weights & Biases, Hugging Face Hub
Analysis	Jupyter, matplotlib

12. How to Run the Full Pipeline

```
uv sync
uv run python data/scripts/download_data.py
uv run python data/scripts/clean_data.py
uv run python data/scripts/convert_to_alpaca.py
uv run python training/validate_dataset.py
uv sync --extra train
huggingface-cli login
wandb login
uv run python -m training.train_unsloth
uv run python -m training.generate_comparison
```

13. Interview Questions and Model Answers

Q: Why QLoRA instead of full fine-tuning?

A: Full fine-tuning of a 3B model needs much more GPU memory and risks overfitting on small domain data. QLoRA trains only adapter weights on a quantized base, making single-GPU training practical while retaining strong task performance.

Q: Why combine three datasets?

A: Each source covers different behavior: conversational follow-ups (HF), categorized technical Q&A (Kaggle), and code reasoning (CodeQA). Combining them creates a multi-skill interview mentor instead of a single-format QA bot.

Q: How did you handle data quality?

A: Multi-stage filtering (null checks, short-answer rejection, deduplication), category normalization, duplicate-field merging, Alpaca validation, duplicate detection in final JSONL, and manual sample review via `sample_records.md`.

Q: How do you ensure reproducibility?

A: Fixed random seeds (42), versioned dataset artifacts in W&B, saved configs (`training_config.json`, `lora_config.json`), and deterministic train/eval split.

Q: What metrics do you track during training?

A: Train loss, eval loss, learning rate schedule, checkpoint steps, and qualitative before/after comparisons on fixed prompt sets across QA, backend, SQL, ML, code, and evaluation tasks.

Q: What would you improve next?

- Increase interview conversation data for better follow-up generation.

- Add RAG over company-specific interview rubrics.
- Use human or LLM-as-judge evaluation with rubric scoring.
- Merge and export GGUF for local deployment.
- Add resume-review task data and evaluation set.

14. Resume Bullet Points (Ready to Use)

- Built a multi-source data engineering pipeline aggregating interview, Q&A, and code datasets into a unified instruction-tuning corpus.
- Implemented preprocessing, deduplication, category normalization, and Alpaca conversion for LLM fine-tuning.
- Designed dataset validation and QA with statistical analysis and W&B dataset artifacts.
- Fine-tuned Llama 3.2 3B Instruct using QLoRA, PEFT, and Unsloth on a custom instruction dataset.
- Reduced trainable parameters by >99% using LoRA while enabling single-GPU training.
- Tracked hyperparameters, checkpoints, and metrics with Weights & Biases.
- Evaluated base vs fine-tuned model across QA, code explanation, and interview evaluation tasks.

15. Glossary

Term	Meaning
SFT	Supervised Fine-Tuning on input-output pairs
LoRA	Low-rank adapter matrices added to frozen layers
QLoRA	LoRA training on 4-bit quantized base weights
PEFT	Parameter-Efficient Fine-Tuning framework
TRL	Transformer Reinforcement Learning library (includes SFTTrainer)
JSONL	One JSON object per line, stream-friendly format
Adapter	Small fine-tuned weight set loaded over base model

Table of Contents

1. Executive Summary (30-Second Interview Pitch)	1
Key numbers to remember	1
2. Problem Statement and Solution Design	2
What the model should do	2
Design decisions	2
3. End-to-End Architecture	2
Folder responsibilities	2
4. Core ML and LLM Concepts You Must Explain	2
4.1 Instruction Tuning	3
4.2 Alpaca Format	3
4.3 LoRA (Low-Rank Adaptation)	3
4.4 QLoRA (Quantized LoRA)	3
4.5 PEFT	3
4.6 Unsloth	3
4.7 SFTTrainer (TRL)	3
4.8 Weights & Biases (W&B)	3
5. Data Sources and Why Each Was Chosen	3
Unified schema (all sources)	4
6. Code Walkthrough: download_data.py	4
Outputs	4
7. Code Walkthrough: clean_data.py	4
Loading logic	4
Cleaning rules	4
Important interview point	5
8. Code Walkthrough: convert_to_alpaca.py	5
9. Code Walkthrough: validate_dataset.py (Day 3)	5
Why validation before training?	5
10. Fine-Tuning Stack (Day 4)	5
config.py	5
prompts.py	6
utils.py	6
train_unsloth.py (primary)	6
train_hf.py (fallback)	6
Training hyperparameter rationale	6
Llama 3.2 chat formatting	6
generate_comparison.py	7
Why not merge adapter into base model yet?	7
11. Inference and Evaluation	7
Six evaluation prompt categories	7

11b. Day-by-Day Project Timeline	7
11c. Technologies Used	7
12. How to Run the Full Pipeline	8
13. Interview Questions and Model Answers	8
Q: Why QLoRA instead of full fine-tuning?	8
Q: Why combine three datasets?	8
Q: How did you handle data quality?	8
Q: How do you ensure reproducibility?	8
Q: What metrics do you track during training?	8
Q: What would you improve next?	8
14. Resume Bullet Points (Ready to Use)	9
15. Glossary	9
Table of Contents	10